LANGUAGE DESIGN / ROUND 2

# Mathematical Objects, Certified Computation

## A unified evaluation of nine proposals for a more natural mathematical proof language

Accord, Cadence, Concord, Facet, Locus,
Meridian, Noema, Prism, and Vantage

---

Prepared for Vladimir Reshetnikov

Codex, with parallel source reviews and independent mathematical checks

6 September 2026 (UTC)

**Reading contract.** This report is self-contained. It explains the inherited design ideas before evaluating their extensions. The cited proposals and earlier syntheses provide provenance and further detail; none is a prerequisite.

## Executive assessment

The nine proposals converge on a useful architectural direction: let mathematicians work with ordinary mathematical objects, and let computations return checked, explicitly limited information about those objects. A polynomial presentation, a finite power-series jet, an interval enclosure, and a symbolic solution set should not all masquerade as equality. Each has its own mathematical contract. This is the strongest advance in the second round.

The practical opportunity is broader than prettier Lean syntax. A mathematician should be able to request a coefficient, choose an induction argument, change representation, or ask for a solution without manually supplying every transport lemma and intermediate witness. Yet the system must preserve the intended statement, expose conditions that affect its meaning, and retain evidence that can be checked after exploratory automation has disappeared. Lean can remain the initial proof backend; a new kernel is neither required nor justified by these reports.

There is substantial agreement on eight broad commitments, with explicit source support in all nine reports: relation-specific computational results; fixed semantic objects and certified presentations or observations; scoped guards; justified execution and reification; theorem-based interfaces separate from search policy; exact-target proof providers; retained replay evidence; and a fair comparison with equally equipped Lean. This is agreement among proposals that share earlier syntheses and examples, not nine independent empirical validations. The substantive differences concern which computation to implement first, how much representation change to automate, and how much evidence the author must see during ordinary work.

The best contributions combine naturally. Facet and Vantage sharpen the meaning of a representation. Noema and Cadence make conditional results and composition precise. Locus supplies a particularly good residual-to-jet theorem. Meridian and Prism develop symbolic calculus with an arbitrary derivative index. Accord emphasizes executable certificates and reusable theorem interfaces. Concord keeps finite computation attached to an arbitrary formal solution. None provides an implemented end-to-end language or a Lean-verified CAS subsystem. All nine Python companions ran successfully in this review, but their finite checks use different units and establish much less than those implementation goals.

I recommend a shared typed request-and-evidence protocol, first exercised by guarded polynomial/rational computation and one formal-series residual problem. Expose the same theorem packages and computation services through ordinary Lean and a small mathematical frontend. Measure whether the frontend improves semantic understanding and maintenance after including infrastructure cost. Keep local differentiation, parameter-degenerate solving, and hostile certificate mutations as discriminating tests. A successful theorem library and linter is a legitimate outcome if additional syntax brings no measurable gain.

## Contents

# 1 Scope, sources, and the meaning of evidence

This report synthesizes the nine packages in `docs/round-2/ideas`, as imported at Git commit `f333ff6cefc8b1f5c9aae79613d10ef95a7ba928`. The current branch was fast-forwarded to that fetched `main` revision before review. All nine TeX reports, their mathematical examples, companion programs, recorded results, and provenance notes were reviewed. Three parallel reading lanes each covered three complete reports; the synthesis also directly inspected the central definitions and mathematical cases. Appendix A provides a source crosswalk, and the adjacent JSON registers provide exact file hashes and one-based TeX line locators.

Two first-round syntheses are part of the intellectual provenance: the earlier Codex report and the Claude report, including its Leant discussion and subsequent questions [10, 11]. Their relevant concepts are explained in Section 2; their questions are restated and assessed in Appendix B. The new report does not require their vocabulary or argument to be remembered.

## 1.1 What was reproduced

The original input packages were left unchanged. Their Python companions were run with bytecode generation disabled and output redirected into this synthesis's evidence directories. All nine completed successfully under Python 3.14.4. Comparisons with shipped results are recorded per package; interpreter metadata can differ without changing the arithmetic result.

| Report | Fresh companion result | What the count means |
|---|---|---|
| Accord | 25 tests | Unit-test methods covering several certificate families. |
| Cadence | 267 checks | Reported finite arithmetic and negative examples. |
| Concord | 459 checks | Includes 441 binomial instances and seven negative cases. |
| Facet | 1,100 checks | Finite checks across its mathematical families. |
| Locus | 24 tests | Includes 702 binomial instances and jets through order 24. |
| Meridian | 27 tests | Reported exact-arithmetic test cases. |
| Noema | 25 tests | Includes 182 binomial instances inside one test method. |
| Prism | 2,475 checks | Reported finite instances; certificate output reproduced. |
| Vantage | 3,013 checks | Reported instances in ten groups. |

These figures must not be added into a proof count or used to rank languages. A thousand evaluations of one identity and a test of one malformed certificate answer different questions. Several producer and checker prototypes share arithmetic code. Reproduction establishes that the supplied executable examples behave as reported in this environment; it does not establish a sound compiler, a verified checker, a universal theorem, or improved author productivity.

## 1.2 Four distinct kinds of support

Throughout the report, *proposed* means an interface or policy is described; *mathematically justified* means an argument with stated hypotheses is given; *executed* means a particular program was run on specified inputs; and *kernel checked* means an actual proof was accepted by the stated Lean environment. These descriptions are not interchangeable. The focused Lean experiments in the accompanying evidence establish only their listed declarations, not the larger language design.

Leant and ProveIt motivate important examples. This synthesis uses the reports' source-attributed observations and the prior review as historical evidence; it does not present those observations as a fresh audit of every file in today's external checkouts. Any new Lean experiment is isolated in the synthesis directory and uses existing dependencies read-only. No external repository is modified or rebuilt for this report.

Some input READMEs refer to checksum artifacts absent from the imported tree. The source register therefore records independently computed hashes of the files actually present. Missing packaging artifacts are not silently reconstructed, and source-manifest assertions are not elevated into fresh validation receipts.

## 2 The problem and the inherited design, explained afresh

Lean's value is that a compact trusted checker can validate explicit proof terms in a precise foundational language. Its elaborator reconstructs omitted information, resolves notation and instances, and translates tactics into terms. A mathematician, however, usually organizes an argument by objects, useful representations, intermediate claims, and reasons. Much formalization work lies between those two levels: finding the correct theorem, aligning its hypotheses, transporting statements between equivalent representations, and exposing facts that paper mathematics leaves locally understood.

These are several different problems. A short proof can conceal a changed statement. A long proof can be a reusable general theorem rather than accidental verbosity. A missing automation rule may require a library improvement, while an unclear scope or hidden domain change may require a language or interface improvement. A sound evaluation must identify which cost it actually removes.

### 2.1 A mathematical step with a checked reason

Consider an author writing, "Reindex the sum by $k = j + i$, then use the binomial theorem." A useful interface should retain both claims and their reasons. It can elaborate the first step using a theorem about a bijection of finite index sets, with explicit obligations for bounds, injectivity, surjectivity, and summand equality. Routine arithmetic can solve those obligations. The author's reason then constrains the accepted derivation: an unrelated global automation success should not pretend to validate the advertised reindexing.

The economical implementation is often an *annotated theorem interface*. An ordinary Lean theorem supplies the logical contract. Additional metadata names semantic roles such as source set, target set, map, and transport proof. Separate policy chooses search order, display defaults, and routine solvers. This division lets the theorem remain useful in ordinary Lean, and prevents search priorities from becoming mathematical axioms. Stable wrapper declarations and role identifiers can insulate the interface from upstream binder renaming, but compatibility still needs checking after refactoring.

A larger *method ledger* records the tree of accepted steps: requested claim, selected method, generated obligations, their proofs, and dependencies. It can support readable explanations and repair. It is not automatically needed for every first prototype. Start with a small theorem interface and retain the data required by a specific user-facing function; do not build an entire method language before demonstrating its value.

### 2.2 Statements, routine work, and definitions

A *statement seal* is a readable account of what the system has actually elaborated: quantified variables, their types, active hypotheses, ambient algebraic structure, domain and branch conventions, and significant implicit choices. It is useful because a kernel proves the elaborated proposition, whereas the author intended a proposition expressed through notation and context. Those can diverge without any kernel error. A seal is an inspectable interface and linting boundary, not a theorem that the author meant the result.

*Routine profiles* specify which obligations automation may discharge without distracting the author. Examples include linear arithmetic, a fixed normalization procedure, or a named simplification set. They need explicit resource bounds, reproducible package identities, and a policy for changed outcomes. "Routine" is a project convention that must be measured on realistic work; it is not synonymous with mathematically trivial or computationally cheap.

Definition design deserves the same attention as proof steps. A definition package may supply an ordinary semantic definition, characterizing equations, existence and uniqueness theorems, executable presentations, observations, and transport laws. Introducing an arbitrary object satisfying a specification is different from constructing one algorithmically, and both differ from selecting one by noncomputable choice. A frontend may make these activities concise, but must preserve the distinction and generate genuine obligations rather than assuming the desired laws.

### 2.3 Suggestions and retained evidence

An automatic provider proposes a term or tactic for an exact goal in an exact context. Acceptance requires validation at that target. The displayed tactic must itself be replayed after insertion: a search tactic may work while the shorter text it prints fails, leaves goals, or affects surrounding syntax. Successful elaboration with an admitted hole does not establish a complete proof. Search failure, resource exhaustion, and a mathematical refutation are different outcomes.

During exploration the system may search widely. Publication should retain the accepted terms or certificates, their source interpretation, and the environment needed for deterministic checking. A script that rediscovers a proof, a saved proof term, and a compiled artifact serve different replay purposes. Upgrading a library is a new checking event with a migration receipt; a previously valid artifact is not evidence that migration succeeded.

These principles explain the second round's starting point. Its important new question is how to extend them from proof steps to computation over mathematical objects without losing domains, precision, or the author's chosen object.

## 3 Where the nine reports converge

Appendix A and `convergence.json` document eight broad commitments, each explicitly proposed by all nine reports. The feature definitions are deliberately precise about their breadth. "Separate verified algorithms from execution," for example, is a design commitment, not a claim that every report gives equally detailed executable semantics.

| | Shared commitment | Practical implication |
|---|---|---|
| R | Relation-specific result | Record equality, domain equality, jet agreement, enclosure, witness correctness, or coverage as different specifications. |
| O | Fixed semantic object | Computable presentations and observations refer to the original object; a convenient replacement needs a theorem. |
| G | Scoped guards | Carry conditions through dependent composition; do not silently narrow the requested domain or add a hypothesis. |
| V | Verified execution boundary | Prove the checker or algorithm, justify its actual execution, and connect reified syntax to the original expression. |
| D | Theorem and definition packages | Keep logical contracts usable in Lean; version search and presentation policy separately. |
| L | Exact-target providers | Replay the actual edit, inspect completion, and avoid promoting abstract search failure to a negation. |
| P | Retained publication evidence | Check accepted assertions from retained evidence in a pinned environment without renewed discovery. |
| E | Fair empirical evaluation | Equip Lean with the same services; include negative neighbors, maintenance cost, and an explicit stopping decision. |

This convergence supports a common prototype architecture. It does not settle the interface. An exact object plus a relation is a strong semantic foundation, but it can support a notebook, a textual proof language, a structured document editor, or Lean commands. Likewise, retaining proof evidence does not determine whether authors usually see a one-line result or an expanded obligation tree.

The agreement is also correlated. The reports share the same two earlier syntheses and many ProveIt examples. Several deliberately preserve the same adversarial boundaries. That is useful refinement, but it makes a nine-way vote a poor measure of independent confidence. A genuinely new round should include held-out mathematical work and readers who did not help design the notation.

## 3.1 Distinctive contributions worth preserving

**Accord** is strongest as a disciplined certificate architecture. It separates mathematical answer kinds from validation states, gives reification a dedicated role, and connects theorem interfaces to publication checks. Its radical-denesting development makes branch selection part of a general contract rather than a successful numerical identity. Its initial slice, finite reindexing plus polynomial certificates, is conservative and useful, although by itself it could be delivered mostly as a Lean library [1, §§5–8,12–17].

**Cadence** emphasizes the composition of guarantees: finite precision, conditional simplification, and complete solution sets. The all-parameter equation $ax = b$ is a particularly good first example because specialization can change both existence and the number of solutions. Cadence asks whether actual execution is justified, not merely whether the source algorithm has a correctness theorem. Its broad agenda makes adherence to its small first slice important; numerical evaluation gates should follow a pilot and precede the main comparison [2, §§4–7,9–10,15–18].

**Concord** makes finite residual certificates central. It shows how a polynomial can establish a finite observation about an arbitrary fixed-point series without changing that series into a canonical implementation. Its separation of nonderivability from negation is essential to provider design. The first slice is mathematically distinctive, but the general bridge from formal residuals to observations still needs Lean implementation [3, §§4–7,10,13,15,17].

**Facet** usefully observes that an exact decoding map does not require a general encoder. A noncomputable semantic object may still have a certified presentation supplied by a producer. Observation handles distinguish a finite answer from a replacement object, and its Abel example maintains that distinction under general algebraic assumptions. The main implementation risk is allowing the generality of the presentation framework to precede a demonstrated workflow [4, §§4–6,9,11–15].

**Locus** supplies the clearest small theorem that links symbolic computation to an infinite semantic object: a sufficiently small residual and a unit derivative identify a finite jet of an existing formal root. This gives a concrete correctness boundary for truncation and Newton refinement. Its dyadic germ is a promising first vertical slice. Existence of the original root and execution of the truncation algorithm remain separate proof obligations [5, §§4–8,12–14].

**Meridian** is especially careful about partial-function domains, substitution guards, and solution predicates. Its Lambert derivative family connects a symbolic index, polynomial arithmetic, analytic differentiation, and local hypotheses. It also warns that an injective map such as $\mathbb{Z} \to \mathbb{Q}$ does not reflect every richer algebraic claim. This is an excellent stress test for transport metadata. Meridian itself chooses partial-rational computation and domain-preserving solving first, with Lambert calculus as a second slice [6, §§4–9,12–16].

**Noema** provides the most compact account of relation-bearing requests and sufficient guards. It does not promise weakest guards or a universal ordering of result types. Its executable cancellation example binds evidence to the request, revision, scope, and required denominators. Stable semantic role identifiers give a practical method-interface design. The prototype is a useful boundary model, not an implementation of dependent contexts or proof checking [7, §§4–8,10–12].

**Prism** combines a strong symbolic-calculus case with demand-driven precision. It also makes an ergonomic point worth retaining: an author may delegate witness construction by specification; safety does not require asking permission for every discovered formula. The accepted witness still needs the same contract and provenance. The implementation challenge is to preserve these precise semantics without making every ordinary request resemble a protocol configuration form [8, §§5–7,9–11,13–18].

**Vantage** offers the sharpest taxonomy: exact presentations, restricted observations, and directed abstractions are different tools. Its normalized EGF interface, developed through Touchard, Bell, and Spivey identities, makes definition and observation design concrete. It also gives a decisive logical example of why failed intuitionistic search is not negation. Some illustrative contracts still need edge hypotheses, including ordered bounds for its telescoping example; this reinforces the value of negative neighbors [9, §§3,5,9–15].

## 3.2 What the convergence does not establish

No report demonstrates that a new input language beats a good theorem library with an equally capable computation service. None settles the cost of checking large certificates or reifying realistic expressions. None measures whether authors notice a weakened result or correctly interpret a compact display. The reports supply enough agreement to stop inventing nine separate architectures; they do not supply enough evidence to freeze the surface language.

# 4 A common semantic model

The common design can be expressed without committing to a new grammar. A request identifies a mathematical context $\Gamma$, an object $x$, an operation or observation, and a required specification. A producer returns data $y$, proposed supporting evidence, and any remaining conditions. Acceptance requires a proof of the requested relation at the original target.

For a conditional transformation, the essential judgment is

$$\Gamma \vdash p : G \longrightarrow \mathcal{R}(x, y).$$

It becomes an unconditional fact in $\Gamma$ only when the guard $G$ is proved there. The guard may depend on the input, the output, and prior choices. The relation $\mathcal{R}$ is part of the mathematical specification, not a display tag attached after a computation has finished.

Sometimes the result itself depends on guard evidence. The dependent type $\prod_{h:G} \sum_y \mathcal{R}(x, y)$ allows such a witness-dependent result; it is not the same interface as $\sum_y \prod_{h:G} \mathcal{R}(x, y)$, which selects one result before receiving the guard proof. Facet makes this scope distinction explicit [4, §5.2]. A general request protocol must preserve it even if the first prototype supports only simpler fixed-output cases.

## 4.1 Three representation relationships

An *exact presentation* consists of data $r$, a meaning map $[\![r]\!]$, any validity condition, and a proof that its meaning is $x$. There need not be a computable function that encodes every semantic object. For example, a polynomial representation of a function may be supplied and proved correct even when the function was introduced abstractly.

A *restricted observation* supplies only some requested information: $r$ agrees with $x$ on a domain, computes its first $N$ coefficients, or encloses a value. It is generally not legitimate to replace $x$ everywhere by $r$. Two observations need only be coherent where their guarantees overlap. Different valid enclosures can have different widths; neither needs to equal the other as an interval.

A *directed abstraction* maps a problem into a simpler language for a particular inference. Its correctness theorem determines which conclusions transport in which direction. A sound prover for an abstraction may establish original claims if each accepted proof reconstructs correctly; its failure does not automatically establish original impossibility. Exact decoding, observational adequacy, and abstract-search adequacy are separate contracts.

## 4.2 Kinds of computational answer

| Answer | Required guarantee | A tempting invalid promotion |
|---|---|---|
| Exact transformation | $x = y$ in the specified structure | Equality after a noninjective map becomes equality before it. |
| Equality on a domain | $\forall u \in D,\ f(u) = g(u)$ | Equality on $D$ becomes global equality, or $D$ silently shrinks. |
| Finite jet | Coefficients agree for every $k < N$ | Finite agreement becomes a formal identity or analytic error estimate. |
| Enclosure | $\ell \le x \le u$, with stated endpoint convention | A floating midpoint becomes an exact value. |
| Witness | $P(y)$ | One root becomes every root. |
| Complete solutions | $\forall y,\ P(y) \leftrightarrow S(y)$ | Generic parameter assumptions disappear under specialization. |
| Factorization | Product identity plus requested factor properties | Product equality becomes irreducibility or uniqueness. |

Exact equality of partial functions additionally requires identical definedness domains and equal values there; equality on a shared subset is weaker. There is no useful universal "confidence level" ordering these answers. A complete solution predicate and an accurate enclosure can be incomparable. Even where guarantees admit weakening, the weakening needs an actual theorem: precision $N$ implies precision $M \le N$, while an interval bound implies a looser interval only after endpoint inequalities are established.

## 4.3 Composition is dependent and directional

Suppose a first stage yields $y$ with $G_1 \to \mathcal{R}_1(x, y)$, and a second stage yields $z$ with $G_2(y) \to \mathcal{R}_2(y, z)$. A composition theorem may establish $\mathcal{R}(x, z)$ under both guards. The intermediate $y$ must be bound before $G_2(y)$ is formed. An implementation cannot flatten these conditions into unrelated strings or move them across a witness scope without justification.

Equality on domains composes on their intersection. That does not authorize answering a request for equality on $D$ with equality on a smaller set: the system must prove that the required $D$ lies in the available domains, branch the task, or report a conditional result. Numerical error composition needs triangle inequalities or stability bounds. Differentiation requires a local equality theorem rather than arbitrary pointwise replacement.

Guards should initially be *sufficient*, not advertised as weakest. A method may require a nonzero denominator although another argument avoids it. Failure to prove that guard is failure of the selected route, not refutation of the requested conclusion. This modest promise is both useful and implementable.

### 4.4 An author-facing request

The following is illustrative syntax, not an implementation claim:

```
Let Delta be a formal series over Q such that
  Delta(0) = 0 and Delta + 4 Delta^2 = 4 t / 9.

Compute the coefficients of Delta below degree 6
  using its defining equation.

Accept J as the degree-below-6 observation of Delta
  because the residual vanishes below degree 6
  and the derivative at the constant term is a unit.
```

The author selects an object, a finite observation, and a reason. A producer may discover $J$ automatically. Acceptance requires the residual certificate and the generic theorem connecting it to $\Delta$. The interface need not ask the author to approve every coefficient. It must show that the result is finite coefficient information and must not replace the original quantified object.

## 5 How a CAS result becomes checked mathematics

A computer algebra system can help find normal forms, factorizations, roots, closed forms, or certificates. The design question is where its authority ends. The reports agree that a plausible output should not acquire mathematical authority merely because it came from a powerful engine.

### 5.1 Three routes, with different engineering costs

**A verified algorithm** comes with a theorem $\forall i,\ \mathrm{Spec}(i, \mathrm{alg}(i))$. To justify a particular returned $o$, the implementation still needs a trusted evaluation of $\mathrm{alg}(i) = o$, or another proved bridge from the actual execution. The theorem about the source function alone does not validate an unchecked native executable's output.

**An untrusted producer with a proved checker** returns $(o, c)$, where $c$ is a certificate. A theorem has the form

$$\mathrm{check}(i, o, c) = \mathsf{true} \implies \mathrm{Spec}(i, o).$$

The actual checker evaluation must also be justified. This route can permit a sophisticated external search while retaining a small checker, but certificate size, checking cost, and reification cost can dominate the apparent benefit.

**A proof-producing transformation** returns a proof term or a sequence of justified transformations. The backend checks those steps against the original goal. This can reuse established tactics and theorem libraries, but a large proof trace may be expensive and fragile under representation changes.

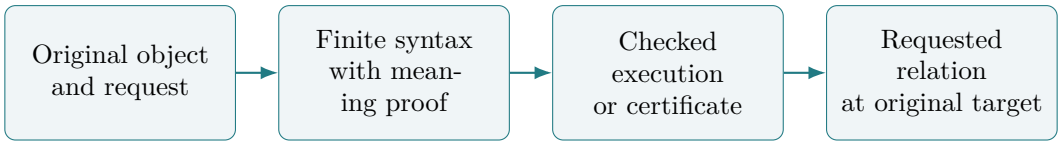
These routes are complementary. A verified polynomial normalizer can check a residual discovered by an untrusted solver; a library theorem can lift the residual into an analytic or formal-series conclusion. The skeptical CAS-integration pattern has substantial prior history, including Harrison and Théry's HOL–Maple work [12]. The present design opportunity lies in exposing such boundaries coherently to mathematical authors, not in claiming that certificate checking itself is new.

### 5.2 The three links that must not be omitted

First, *reification* must prove that the finite syntax passed to the checker denotes the original mathematical expression. Parsing the wrong coefficient domain or erasing a denominator condition can invalidate this link while the arithmetic checker remains perfectly sound.

Second, *execution* must establish the actual evaluation needed by the soundness theorem. Kernel reduction, checked proof reconstruction, and a native evaluation mechanism can have different trusted dependencies. Lean's proof validation documentation makes the distinction between kernel validity, interpretation, axioms, and execution assumptions explicit [13]. A publication policy should state the accepted trust profile for its pinned version; a generic "verified" badge is insufficient.

Third, *reconstruction* must use the result at the caller's original target, with the same semantic object, scope, and requested guarantee. Proving an unrelated normalized polynomial identity is not enough. These links form a chain; stronger checking in one link cannot repair a missing link elsewhere.




### 5.3 Keep computation out of definitional equality initially

Definitional equality is the conversion relation used throughout elaboration and type checking. Adding broad CAS normalization there would make ordinary checking depend on expensive, domain-sensitive computation and complicate both predictability and trust. The proposed computations should initially produce ordinary theorems at explicit request boundaries. A result can then rewrite a goal through a proof, leaving foundational conversion rules unchanged.

This is also a maintenance advantage. A changed heuristic can produce a different certificate or fail to find one without changing what counts as the same type throughout the language. Logical interfaces should be stable before search policy is made ambitious.

## 6 Worked case I: guards, domains, and complete answers

The small rational identity

$$\frac{x^2-1}{x-1} = x+1$$

is an unusually effective language test. As an equality of real expressions using total division, it needs $x \neq 1$. At $x = 1$, the left side is zero and the right side is two. As an equality of rational functions, it has a different interpretation. As equality of partial functions, both values and domains must be compared. A frontend must not slide between these meanings because they share a familiar printed fraction.

The correct global total-function identity is piecewise:

$$\frac{x^2-1}{x-1} = \begin{cases} 0, & x = 1, \\ x+1, & x \neq 1. \end{cases}$$

An interface can offer a conditional equality, this global piecewise result, or a domain-restricted transformation, depending on the request. Those are different mathematical answers with different usefulness. Noema's prototype models retention of the original denominators and rejection of a certificate with substituted guards; Cadence, Meridian, and Vantage generalize the distinction to guarded computation and coverage [7, §§5–6,11.1][2, 6, 9].

Complete solving adds a second dimension. Over a field, the equation $ax = b$ has the following complete description:

$$\{x : ax = b\} = \begin{cases} \{b/a\}, & a \neq 0, \\ \text{the entire field}, & a = 0 \text{ and } b = 0, \\ \varnothing, & a = 0 \text{ and } b \neq 0. \end{cases}$$

Verifying $b/a$ under $a \neq 0$ is witness correctness. Proving the displayed classification requires soundness and coverage for all branches. A generic solution formula cannot be specialized to $a = 0$ as though the original nonzero guard had vanished. A *solution predicate* is often the right semantic interface, because some solution sets are infinite and cannot be represented by a finite list.

Branch selection appears even in elementary radical denesting:

$$\sqrt{5 + 2\sqrt{6}} = \sqrt{3} + \sqrt{2}.$$

The square identity is necessary, but the nonnegative branch condition is also needed. The negative of the proposed right side has the same square and is therefore an excellent malformed candidate. Similarly, isolating the positive root of $x^2 - 2$ does not describe all roots, and a real square-root statement does not automatically transfer to a complex principal-branch statement.

**Recommendation.** Implement the guarantee selector as a mathematical part of the request: equality on this domain, one witness, or a complete solution predicate. Let computation supply candidates, and make a weaker answer visible as a different result that cannot discharge the stronger request.

## 7 Worked case II: exact finite information about an infinite object

Let $R[[t]]$ denote formal power series over a commutative ring $R$. Write

$$A \equiv_N B \quad \Longleftrightarrow \quad [t^k]A = [t^k]B \quad \text{for all } k < N.$$

Equivalently, $A - B$ belongs to the ideal $(t^N)$. This is exact finite information, often called a jet. It is not a floating approximation, an analytic big-O statement, or an assertion that the full series are equal.

Addition and multiplication preserve the minimum available precision. Differentiation generally consumes one order: input precision $N + 1$ suffices for output precision $N$. For example, $t$ and $t + t^3$ agree below degree three over $\mathbb{Q}$, while their derivatives differ at degree two. Substitution has its own side conditions; substituting a series with nonzero constant term into an arbitrary formal series is not generally defined by finite coefficient sums. Precision planning must invoke the appropriate operation theorem, not apply one universal arithmetic rule.

## 7.1 A general residual-to-jet theorem

The following theorem captures Locus's central idea and makes its hypotheses explicit [5, formal-root development].

**Proposition 1.** *Let $F(Y) \in R[[t]][Y]$, and let $\Delta \in R[[t]]$ satisfy $F(\Delta) = 0$ and $\Delta(0) = d$. Reduce the coefficients of $F$ modulo $t$ to obtain $\overline{F} \in R[Y]$. Assume $\overline{F}'(d)$ is a unit in $R$. If $N \geq 1$, $J(0) = d$, and $F(J) \equiv_N 0$, then $J \equiv_N \Delta$.*

*Proof.* Polynomial subtraction factors as

$$F(J) - F(\Delta) = (J - \Delta)U$$

for a series $U$. Because $J$ and $\Delta$ have the same constant term, $U(0) = \overline{F}'(d)$. A formal series with unit constant coefficient is a unit. Thus $J - \Delta = F(J)U^{-1}$ belongs to $(t^N)$. □

No field assumption is needed. Conversely, "nonzero" cannot replace "unit" over an arbitrary ring. In $R = \mathbb{Z}/4\mathbb{Z}$, take $F(Y) = 2Y$, $\Delta = 0$, and $J = 2t$. The residual is exactly zero and the derivative is the nonzero element 2, but $J \not\equiv_2 \Delta$. A theorem wrapper that forgets the unit condition would admit a false observation.

The theorem is relative to an *existing* root. It neither constructs that root nor proves its existence. This is a feature: an author can retain an arbitrary $\Delta$ from a theorem statement while a finite checker computes information about it. A separate existence theorem can be supplied when the task actually introduces a new root.

## 7.2 A concrete dyadic germ

Take the rational formal series specified by

$$\Delta + 4\Delta^2 = \frac{4}{9}t, \qquad \Delta(0) = 0.$$

Writing $\Delta = \sum_{n \geq 1} a_n t^n$ gives

$$a_1 = \frac{4}{9}, \qquad a_n = -4 \sum_{i=1}^{n-1} a_i a_{n-i} \quad (n \geq 2).$$

The first five coefficients give

$$J = \frac{4}{9}t - \frac{64}{81}t^2 + \frac{2048}{729}t^3 - \frac{81920}{6561}t^4 + \frac{3670016}{59049}t^5.$$

A rational polynomial checker can establish that $J + 4J^2 - 4t/9$ has zero coefficients below degree six. The derivative of the defining polynomial at the constant term is 1, hence a unit. Proposition 1 then gives $J \equiv_6 \Delta$. The checker's job is finite arithmetic; the library theorem handles the infinite object. The coefficient formula

$$a_n = (-1)^{n-1} \frac{4^{2n-1}}{9^n} C_{n-1}, \qquad n \geq 1,$$

where $C_m$ is the $m$-th Catalan number, is a separate universal result obtainable from the recurrence. Checking the first five coefficients does not prove that formula for every $n$.

Newton refinement fits the same contract. If $F'(J)$ is a unit and $F(J) \in (t^N)$, put $h = -F(J)/F'(J)$. Then $h \in (t^N)$, and the polynomial identity

$$F(J+h) = F(J) + F'(J)h + h^2 H$$

shows the new residual lies in $(t^{2N})$. A finite implementation still needs a proof that its truncations and inverse computation realize these operations to the advertised precision. The algebraic doubling argument does not supply that execution proof automatically.

### 7.3 A distinct route: precision-improving fixed points

Concord uses a related but different mechanism [3, §7.3]. Over a commutative $\mathbb{Q}$-algebra, let $\Psi(S) = z\exp(-aS)$ on zero-constant formal series. If $S \equiv_k T$, then $\Psi(S) \equiv_{k+1} \Psi(T)$. For an arbitrary fixed point $T = \Psi(T)$, a candidate $P$ with zero constant term and $P - \Psi(P) \equiv_N 0$ therefore satisfies $P \equiv_N T$: start with constant-term agreement and improve it one order at a time up to $N$. For instance,

$$P = z - az^2 + \frac{3}{2}a^2z^3 - \frac{8}{3}a^3z^4$$

is the expected observation below degree five. This is a contraction argument in formal precision, not a numerical convergence theorem. It illustrates why a general observation interface should admit multiple mathematically certified routes without forcing them into one algorithm.

**Recommendation.** Use residual-to-observation as the first nontrivial bridge between a CAS and an arbitrary semantic object. Keep precision in the type of the result, and evaluate demand-driven precision as a separate optimization with its own correctness and performance measurements.

## 8 Worked case III: algebraic generality and symbolic calculus

Conciseness should not be purchased by silently strengthening hypotheses. A computer algebra routine may prefer fields, while the theorem being proved is about an additive group or a ring with zero divisors. The language needs to preserve that distinction and make legitimate transport reusable.

### 8.1 Binomial inversion and quantifier scope

For sequences in an additive commutative group, define the binomial transform using integer scalar action. The correct inversion statement is

$$\left(\forall n, \quad b_n = \sum_{k=0}^{n} \binom{n}{k} \cdot a_k\right) \iff \left(\forall n, \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} \cdot b_k\right).$$

Here the bold dot means scalar action, not multiplication of two elements of the group. The scalar orthogonality identity is proved over $\mathbb{Z}$, then applied through distributivity and finite-sum rearrangement. No field structure on the sequence values is needed.

The scope of $\forall n$ matters. At a single fixed index, the displayed forward equality does not imply the inverse equality at that index, because the latter depends on earlier rows. Take $n = 1$, $a_0 = a_1 = 0$, $b_0 = 1$, and $b_1 = 0$. The forward row says $0 = 0$; the inverse row would say $0 = -1$. Cadence's and Concord's compact displays need to be read with the uniform sequence-level quantification supplied by their surrounding argument [2, §8.3][3, §8.3]. The focused Lean examples in the evidence package check both the valid forward row and failure of its supposed pointwise inverse.

This is a statement-interface issue as much as a theorem issue. A compact renderer that moves or hides a quantifier can change the claim. The statement seal should expose the uniform scope, while the method interface records that all lower-index equations are available to the inversion proof.

## 8.2 Transport is a theorem about a particular relation

A polynomial identity over $\mathbb{Q}$ evaluates correctly in every commutative $\mathbb{Q}$-algebra, including the dual-number ring $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$. The forward preservation theorem does not require an injective scalar map. Thus a universal polynomial computation can support a theorem with zero divisors, provided its certificate uses only valid rational-algebra operations. Trying a few rational substitutions does not establish that universal identity.

Reflection is a different direction. An injective map can reflect equality of mapped elements, but not every statement about a richer target structure. For example, $X$ belongs to the ideal generated by $2X$ in $\mathbb{Q}[X]$, using the multiplier $1/2$. It does not belong to the corresponding ideal in $\mathbb{Z}[X]$: the coefficient of $X$ in $2Xh(X)$ is even for every integer polynomial $h$. The embedding $\mathbb{Z} \hookrightarrow \mathbb{Q}$ is injective; the witness domain has nonetheless expanded. A representation registry must attach transport theorems to individual relations rather than advertise a universal "safe coercion" flag.

The formal Abel identity illustrates the positive opportunity. If an arbitrary zero-constant series satisfies $T = z\exp(-aT)$, then over a commutative $\mathbb{Q}$-algebra the normalized exponential-generating-function observation is

$$n!\,[z^n]\exp(xT) = x(x - na)^{n-1} \qquad (n \geq 1),$$

with constant coefficient 1. A universal coefficient theorem and polynomial transport can establish this under general hypotheses. A finite jet of one canonical solution cannot establish it for every $n$ and every originally quantified solution. Facet and Concord are particularly useful on that object identity boundary; Vantage develops normalized EGF observations into a broader definition interface [4, 3, 9].

## 8.3 A symbolic derivative family with local hypotheses

Meridian and Prism develop a stronger example than evaluating a few derivatives [6, 8, Lambert derivative developments]. Let

$$P_0(w) = 1, \qquad P_{n+1}(w) = (1 + w)P_n'(w) - \big((n + 1)w + 3n + 2\big)P_n(w),$$

so every $P_n$ is an integer polynomial. The first terms are

$$\begin{aligned} P_1(w) &= -w - 2, \\ P_2(w) &= 2w^2 + 8w + 9, \\ P_3(w) &= -6w^3 - 36w^2 - 79w - 64. \end{aligned}$$

Set

$$H_n(w) = \frac{e^{-(n+1)w} P_n(w)}{(1+w)^{2n+1}}, \qquad \phi(w) = \frac{e^{-w}}{1+w}.$$

For $w \neq -1$, the product and quotient rules give

$$H_n'(w)\phi(w) = H_{n+1}(w).$$

The displayed recurrence is exactly the polynomial numerator remaining after common factors are extracted. Its index starts at zero here; some source presentations shift the polynomial index by one.

Suppose $U \subseteq \mathbb{R}$ is open and $W : \mathbb{R} \to \mathbb{R}$ has derivative $\phi(W(x))$ at every $x \in U$, with $W(x) \neq -1$ there. Then

$$W^{(n+1)}(x) = H_n(W(x)) \qquad (x \in U)$$

follows by induction, using the chain rule on the entire open set. Derivative existence is part of the hypothesis and induction, not inferred from an equation involving a totalized derivative operator alone. The condition $W(x) \neq -1$ is the relevant pole exclusion; imposing $W(x) > -1$ would needlessly exclude another real branch in applicable settings.

The neighborhood condition is essential. Two functions can agree at a point and have different derivatives there: $f(x) = x$ and $g(x) = 0$ agree at zero. An induction hypothesis valid on $U$, together with openness, yields the local equality needed to differentiate. A proof method should package this analytic plumbing as an ordinary theorem, while the computation service certifies the polynomial recurrence. Checking only $P_0, \ldots, P_3$ does not prove the arbitrary-index recurrence or the analytic induction.

The underlying recurrence and analytic argument are attributed by the proposals to existing ProveIt work. The language contribution is a usable interface between a computable finite representation and the general theorem, not a claim to have discovered the mathematical family. This makes a good staged integration test after a smaller polynomial and formal-jet slice is working.

### 8.4 A small specification gap with a useful lesson

Vantage's illustrative telescoping contract should make its range convention explicit. For inclusive natural intervals and $a \leq b$,

$$\sum_{k=a}^{b} (G(k+1) - G(k)) = G(b+1) - G(a).$$

If $a = 3$, $b = 1$, and $G(k) = k$ in $\mathbb{Z}$, the sum is empty and equals zero, while the displayed right side is $-1$. An ordered-range premise or a correct empty-range branch is needed [9, §10.4]. This is a gap in an illustrative proposed contract, not an implemented checker defect. Such tiny negative neighbors often test a language's semantic discipline better than another successful long example.

## 9 Leant, proof providers, and negative evidence

The user's Leant work on automatic term synthesis and tactic suggestion is directly relevant to this architecture. The earlier source review distinguishes search over a restricted term language from Lean-side acceptance, and distinguishes a successful search invocation from successful replay of the text shown to the user [10, 11]. The second-round proposals largely accept those boundaries and

extend them to CAS and counterexample providers. The analysis below is a proposed integration contract, not a claim that the current external Leant checkout already implements every field.

## 9.1 A provider supplies candidates for an exact request

A request should carry, or refer unambiguously to, the elaborated target, ordered local context, universe and instance information, relevant environment identity, source revision, and policy. A provider may search using a simpler internal representation, but an accepted proof must reconstruct at that exact request. A hash helps identify a request; it is not a proof that the encoded context faithfully represents the caller's semantics.

The provider can return a candidate term, a tactic edit, a certificate, an explicitly conditional result, or no result. These should be decoded through a versioned protocol rather than inferred from a suggestive log line. Resource exhaustion and unsupported input deserve distinct diagnostic explanations even when neither produces mathematical evidence.

For a tactic suggestion, retain three separate receipts:

1. The search or synthesis operation produced a candidate under the stated request and budget.
2. The proposed tactic or term elaborated at the exact target with the required completion and axiom policy.
3. The actual displayed edit was inserted transactionally into the source and replayed successfully in its surrounding context.

The third receipt matters because formatting, name resolution, comments, indentation, or a shortened suggestion can make the displayed edit differ from the successful internal action. A failed attempt must restore metavariable and source state before another candidate is tried. Retaining a term cannot excuse a broken advertised edit; retaining a working edit cannot excuse a changed goal.

## 9.2 Why complete search failure is still not negation

Even a complete search for a specified calculus reports a judgment about that calculus. It does not automatically prove a negation in Lean. A simple example, emphasized by Vantage, is the schema $P \vee \neg P$: it is not uniformly derivable in intuitionistic propositional logic, yet intuitionistic reasoning proves

$$\neg\neg(P \vee \neg P).$$

Indeed, assuming $h : \neg(P \vee \neg P)$, any proof of $P$ contradicts $h$, so $\neg P$; that right-hand disjunct then also contradicts $h$. Thus the lack of an intuitionistic derivation cannot be reinterpreted as $\neg(P \vee \neg P)$. Classical Lean can prove the excluded-middle schema under its classical assumptions [9, §5.7].

A provider may legitimately report "not derivable in this calculus under these assumptions," or supply a checked countermodel with a theorem connecting it to the requested judgment. To refute an original Lean proposition $Q$, it needs an original-target proof of $\neg Q$, or a proved adequacy bridge plus checked evidence sufficient to construct that proof. An empty recorded list of abstracted nodes is diagnostic metadata, not such a bridge. Nor does failure to synthesize a polymorphic term of type $\forall A, A$ mean that every individual $A$ is empty.

This distinction should affect interface wording. "No candidate within this budget," "unsupported target," "not derivable in the stated calculus," and "proved false" are useful different outcomes. The

UI need not expose internal search machinery to every author, but it must not display a stronger mathematical conclusion than the evidence supports.

### 9.3 Term synthesis and CAS can share an acceptance boundary

The common provider abstraction is not "a service that returns proofs." It is "a service that proposes data and evidence for a typed request." A term synthesizer proposes an inhabitant; a CAS proposes a normal form and certificate; a counterexample searcher proposes a witness refuting a statement. The same original-target validation, transaction isolation, provenance, resource accounting, and retained replay rules can apply to all three.

This does not imply one universal certificate language. The mathematical soundness theorem remains specific to each provider and guarantee. A shared envelope reduces duplicated integration work; a shared untyped notion of success would erase exactly the distinctions the reports have clarified.

## 10 Decisions still open, and refinements proposed here

The papers differ more in implementation strategy than in foundational logic. The following choices deserve explicit decisions rather than another layer of compatible terminology.

| Choice | Main benefit | Main unresolved cost |
|---|---|---|
| Guarded algebra first | Small checkers; cancellation and complete linear solving expose semantic mistakes immediately. | May show service and library value without establishing a need for new syntax. |
| Formal-root jets first | Demonstrates computation about an arbitrary infinite object through a compact generic theorem. | Needs formal-series interfaces, precision contracts, and a verified finite representation. |
| Symbolic calculus first | Tests arbitrary indices, local hypotheses, and representation transport together. | Too many moving parts can obscure which component failed or improved usability. |
| Rich definition interface first | Attacks the cost of introducing reusable objects and observations, including EGF packages. | A broad schema language risks preceding evidence about common definition patterns. |
| Document view first | Can test comprehension and statement visibility using existing Lean input. | May leave the authoring burden largely unchanged. |
| New input form first | Directly tests concise mathematical authoring. | Requires an additional interpretation and repair surface before benefits are known. |

### 10.1 Reuse evidence through explicit entailment

Generalizing the reports' observation-weakening rules [3, §7.4][2, §4.3], this synthesis recommends an explicit reuse rule beyond exact hash matching or an informal claim that one result is "stronger." A retained certificate may be reused when a checked adapter shows that the current request follows from its guarantee: the current context proves its guards, the semantic object correspondence is established, and the retained relation entails the requested relation under the current policy.

Different coordinates have different variance. Higher jet precision can answer a lower-precision request. Equality on a larger domain can answer one on a smaller domain. A narrower interval can answer a wider enclosure request. Stronger assumptions make a theorem less broadly applicable, not more. Additional axioms may violate the new project's trust policy even when the proposition is identical. The adapter should prove these individual entailments rather than squeeze them into a single rank.

Exact identity remains an efficient fast path. Entailment-based reuse is an optional extension whose benefit must exceed proof and cache complexity. It should not be used to hide a changed statement during source editing.

## 10.2 Make the visible obligations sufficient to interpret the claim

Readable mathematics requires compression. The right question is which information may be collapsed without misleading the reader. I propose a *visible obligation boundary*: the default rendering must expose unresolved conditions and accepted distinctions that affect the claim's domain, branch, precision, completeness, or trust policy. Routine proofs behind those conditions can be folded away while remaining inspectable.

This is a rendering contract against the typed evidence graph, not an assertion that there is an easily computable minimal display. A finite-jet result should show its precision; a conditional equality should show its condition; a root selection should show which root is selected. An expanded view can explain why those conditions suffice. Testing whether readers reconstruct the intended claim is more informative than counting visible lines alone.

## 10.3 Keep status dimensions separate

An implementation can avoid an unwieldy enumeration of every possible result by separating: proof validation status; unresolved guards; guarantee kind; source/provenance binding; and policy compatibility. A fully checked theorem $G \to Q$ may still have an unresolved guard $G$ at the caller. A valid finite observation may still fail a request for exact equality. A proved result may still need source-edit replay. These are ordinary, explainable states rather than exceptions to a binary success badge.

## 10.4 Route changes should be meaningful edits

Changing an internal polynomial normalization algorithm need not prompt the author if the requested relation, object, and accepted policy remain fixed and the result rechecks. Changing a branch, witness, domain, method explanation, or public theorem dependency can affect mathematical meaning or maintainability. Such changes should be reported at the level of the resulting statement or argument. A blanket ban on automatic choices would defeat the goal of synthesis; invisible semantic changes would defeat its reliability.

Coherence is therefore relative to what the request observes. Two exact presentations can be coherent through their shared object. Two valid intervals need not have identical endpoints. Two proofs of a proposition may be interchangeable for kernel acceptance but differ as explanations of the author's chosen method. Logical equivalence alone does not settle every editorial policy.

# 11 A discriminating implementation and evaluation plan

The next iteration should produce one shared protocol and a small complete workflow. Nine more surface grammars would add less information than one experiment that reveals which layer actually saves effort.

## 11.1 Stage one: a checked computation request

Implement a typed request with source binding, a mathematical guarantee, and scoped guards. Supply an ordinary Lean theorem interface and a finite integer or rational polynomial representation with proved interpretation and checker soundness. Exercise guarded cancellation and all-parameter $ax = b$. Accept a proof only through the actual execution route selected by the project's trust policy. Retain evidence and replay it after disabling the discovery service.

The exit gate is an end-to-end demonstration, not a large example count: the original source request elaborates, the producer is untrusted, malformed certificates are rejected at the correct boundary, the accepted result has the advertised domain and completeness, and saved evidence rechecks without search. Document checker time, certificate size, reification time, and dependency cost.

## 11.2 Stage two: a fixed object and a finite observation

Implement Proposition 1 and an executable truncated-polynomial checker for the dyadic germ. The theorem statement must quantify the original $\Delta$; the computed $J$ supplies only the requested jet. Demonstrate changing the requested precision, changing the producer, and rejecting a nonunit derivative hypothesis in a generalized ring example. Measure where full normalization, certificates, and demand-driven truncation differ in cost.

Use the same service from ordinary Lean and a small mathematical interface. This is the first strong test of whether a new authoring layer improves understanding beyond what the theorem package alone achieves. Keep the initial definition package small: semantic object, defining equation, observations, and the specific transport theorems used in the experiment.

## 11.3 Stage three: semantic integration and a held-out task

Add the symbolic derivative family from Section 8, or an equally demanding case selected before implementation. This tests locality, arbitrary indices, and algebraic representation without merely replaying the first example. Reserve an additional, previously unseen related theorem for the held-out evaluation; a case used to design the system is not held out. Include a definition-interface task such as normalized EGF coefficients and a task involving a complete solution predicate.

Apply controlled edits: rename a wrapper role, change a theorem's hypotheses, alter a branch condition, move a proof step into another scope, change a routine profile, and upgrade a dependency. Record which edits preserve a certificate, which require a checked adapter, and which require new discovery. Do not count automatic repair as successful until the resulting statement, explanation, and actual source replay have been checked.

## 11.4 Separate the effects of syntax, libraries, and services

At minimum compare four treatments:

1. ordinary Lean with the existing library;
2. Lean with the new theorem and definition packages;
3. Lean with those packages and the same computation/provider service;
4. the proposed mathematical frontend with exactly those packages and service.

A rendered document view can be an additional ablation rather than a different backend. Match training time, solver budgets, and available hints. Include authors other than the designers. Report startup learning cost separately from repeated use, and preserve unsuccessful attempts rather than measuring only finished proofs.

Useful outcomes include time to a correctly stated checked theorem; author edits and interventions; maintenance time after a controlled change; reader accuracy about the actual claim; and all amortized infrastructure work. Visible token count can describe presentation, but should not serve as the primary success metric. A ten-line proof that hides the wrong domain has not improved mathematical expression.

## 11.5 A blind semantic-reading test

Before readers see the expanded evidence, ask them to reconstruct the guarantee of a compact result: the quantified object, domain, branch, precision, completeness, and unresolved conditions. Use matched positive and negative neighbors with equally plausible formatting. Compare their answers with the elaborated specification, then reveal the expanded view and measure whether it resolves mistakes. This directly tests whether concise notation communicates mathematics rather than merely looking familiar.

| Negative neighbor | Required system behavior |
|---|---|
| Cancellation at the pole | Reject unconditional equality; preserve the guard or supply a checked piecewise result. |
| Specialize $ax = b$ at $a = 0$ | Recompute branch coverage; do not reuse the generic singleton as a complete answer. |
| Differentiate a finite jet | Require enough input precision and actual local analytic hypotheses when differentiating functions. |
| Nonunit formal derivative | Refuse the residual-to-jet theorem unless its unit premise is established. |
| One binomial row | Refuse the sequence inversion method without all required rows or an appropriate uniform hypothesis. |
| Integer ideal via rational witness | Refuse backward transport without a relation-specific theorem. |
| Wrong radical sign | Reject the candidate despite the matching square. |
| Reversed telescoping bounds | Use a proved empty-range case or reject the ordered-range method. |
| Stale target or sibling scope | Reject evidence at the request-binding boundary. |
| Successful search, broken displayed edit | Keep the source unchanged and report that edit replay failed. |
| Complete abstract search failure | Report the calculus-level outcome; do not manufacture an original-target negation. |

## 11.6 Choose thresholds before the main study

The reports agree on measurement but do not establish numeric success thresholds. Set those after a small pilot determines task variance and before evaluating the main comparison. Report paired results and uncertainty, not just averages. Predeclare a practical improvement target, acceptable semantic-reading error, checking-cost limits, and an infrastructure amortization horizon.

Stop or reduce the language layer if package and service improvements explain the observed gains, if readers systematically mistake weaker guarantees for stronger ones, or if maintenance cost exceeds the saved author effort. A successful Lean library, provider protocol, and linter can be the right result. This stopping rule protects the mathematical objective from commitment to a particular syntax or project identity.

## 12 Questions for the next discussion

The following questions are intended to produce decisions or experiments. They are not requests to answer every architectural detail before coding.

**Q1. Which first task must work from authored source to retained proof?** Choose guarded cancellation and complete linear solving as the smallest slice, or the formal-root jet as the distinctive slice. Name the exact theorem, negative neighbor, and replay artifact required for completion.

**Q2. What must every request say about its result?** Decide the initial guarantee vocabulary: equality, equality on a domain, finite jet, witness, and complete solution predicate are enough to expose most current disagreements. Which can be inferred without surprising the author?

**Q3. What is the initial trusted execution route?** Select one route for polynomial checking, state its pinned axioms and runtime dependencies, and measure reification and proof checking separately from candidate discovery.

**Q4. What can an author delegate by specification?** Distinguish choosing a witness that satisfies a fixed request from changing the requested object, branch, domain, or proof explanation. Which changes require a visible notice, and what evidence allows the notice to be dismissed?

**Q5. How much of a conditional result is visible by default?** Use the blind reading experiment to decide whether a collapsed guard badge, inline condition, or case split communicates the actual claim reliably.

**Q6. What representation changes are automatic?** Begin with exact presentations and requested observations of a fixed object. Decide whether the first version pins routes, permits certified coherent alternatives, or supports entailment-based certificate reuse across changed requests.

**Q7. What precisely can Leant report negatively?** Specify protocol outcomes for unsupported input, bounded failure, complete calculus nonderivability, and checked original-target refutation. Identify the missing adequacy theorem for any proposed promotion between them.

**Q8. Which definition interface is worth implementing first?** Compare the formal-root package and normalized EGF observations. List the ordinary Lean definitions and theorems each needs before choosing new declaration syntax.

**Q9. What evidence survives publication and migration?** Decide which terms, certificates, source mappings, method explanations, and environment pins are mandatory, and what an upgrade must recheck to produce a new receipt.

**Q10. What result would persuade us to keep only the library and tools?** Predeclare the fair Lean baseline, independent-reader task, maintenance edits, cost accounting, and practical stopping threshold. This is the decision that makes the next iteration an experiment rather than another advocacy document.

# A Convergence crosswalk and source locators

The eight feature codes are defined in Section 3. Every cell below is an *explicit proposal*, not an implemented capability. Each cell gives a representative printed section and one-based TeX source range. The machine-readable `convergence.json` retains all 241 supporting references, qualifications, and the feature definitions for the 72 cells. This compact table is a reading aid, not a substitute for those qualifications.

| Report | R/O/G/V source locators | D/L/P/E source locators |
|---|---|---|
| **Accord** | **R**: §6.1, lines 519–549<br>**O**: §5.1, lines 409–420<br>**G**: §8.1, lines 775–788<br>**V**: §7.1, lines 612–648 | **D**: §5.1, lines 409–420<br>**L**: §13.2, lines 1319–1346<br>**P**: §14.3, lines 1448–1463<br>**E**: §17.1, lines 1588–1600 |
| **Cadence** | **R**: §2.3, lines 286–301<br>**O**: §5.1, lines 507–543<br>**G**: §4.2, lines 434–449<br>**V**: §6.2, lines 653–677 | **D**: §5.1, lines 507–529<br>**L**: §12.2, lines 1325–1341<br>**P**: §14.1, lines 1491–1510<br>**E**: §16.1, lines 1611–1630 |
| **Concord** | **R**: §4.1, lines 252–280<br>**O**: §6.1, lines 393–398<br>**G**: §4.2, lines 283–296<br>**V**: §5.1, lines 322–344 | **D**: §4.3, lines 298–309<br>**L**: §12.1, lines 800–805<br>**P**: §15.1, lines 918–923<br>**E**: §17.1, lines 960–977 |
| **Facet** | **R**: §4.2, lines 415–448<br>**O**: §4.1, lines 378–413<br>**G**: §5.2, lines 510–529<br>**V**: §6.1, lines 614–644 | **D**: §5.1, lines 489–508<br>**L**: §11.3, lines 1212–1255<br>**P**: §13.2, lines 1389–1400<br>**E**: §14.3, lines 1501–1538 |
| **Locus** | **R**: §4.1, lines 448–474<br>**O**: §3.1, lines 351–373<br>**G**: §4.5, lines 560–577<br>**V**: §4.1, lines 463–474 | **D**: §5.1, lines 598–620<br>**L**: §11.3, lines 1445–1470<br>**P**: §12.2, lines 1572–1592<br>**E**: §13.3, lines 1741–1772 |
| **Meridian** | **R**: §4.1, lines 444–494<br>**O**: §3.1, lines 308–326<br>**G**: §6.2, lines 739–772<br>**V**: §5.1, lines 568–605 | **D**: §3.4, lines 398–417<br>**L**: §11.2, lines 1492–1516<br>**P**: §12.2, lines 1649–1672<br>**E**: §14.1, lines 1819–1848 |
| **Noema** | **R**: §3.1, lines 155–173<br>**O**: §7.1, lines 401–408<br>**G**: §5.1, lines 268–287<br>**V**: §6.1, lines 334–337 | **D**: §7.1, lines 401–408<br>**L**: §8.4, lines 503–522<br>**P**: §10.1, lines 733–740<br>**E**: §11.2, lines 780–789 |
| **Prism** | **R**: §4.1, lines 235–246<br>**O**: §5.1, lines 292–299<br>**G**: §4.3, lines 266–284<br>**V**: §7.1, lines 396–409 | **D**: §10.2, lines 640–653<br>**L**: §6.3, lines 381–388<br>**P**: §15.3, lines 923–928<br>**E**: §18.1, lines 1023–1026 |
| **Vantage** | **R**: §7.1, lines 894–910<br>**O**: §3.1, lines 352–367<br>**G**: §3.4, lines 422–440<br>**V**: §7.2, lines 912–941 | **D**: §6.1, lines 790–806<br>**L**: §5.7, lines 761–787<br>**P**: §6.5, lines 865–877<br>**E**: §15.1, lines 1962–1973 |

## B Earlier questions, restated and carried forward

The earlier two syntheses posed 22 questions. They are restated here so that this document remains independently readable. The right column gives this report's assessment of the second-round response. "Proposed" is not "empirically settled." S identifies a question from the Codex synthesis; U identifies one from the Claude synthesis [10, 11].

| ID | Restated question | Current answer and remaining test |
|---|---|---|
| S1 | What is the smallest useful mathematical step interface? | Begin with ordinary theorem contracts and semantic role metadata. Add a method ledger only for demonstrated explanation or repair needs. Test reindexing and guarded computation end to end. |
| S2 | Does a written reason constrain the proof? | Yes when it is presented as the accepted justification. Explicitly distinguish a hint from a required method; unrelated automation must not impersonate that method. |
| S3 | Which choices may automation change? | Preserve the requested object and guarantee; classify branch, domain, witness, and explanation changes visibly. Test actual author expectations under edits. |
| S4 | When is a suggestion verified? | Exact-target acceptance, completion and policy checks, and replay of the displayed edit are separate required receipts. An integrated implementation remains to be demonstrated. |
| S5 | How much infrastructure and routine automation is justified? | Bound and version profiles; charge theorem-package, checker, and wrapper work to the evaluation. Numeric cost thresholds await a pilot. |
| S6 | How do views compose and stay coherent? | Use relation-specific composition, scoped guards, and object-relative coherence. Pin routes initially; do not require identical outputs for observations such as enclosures. |
| S7 | How should an elaborated statement be inspected? | Show quantifiers, structures, domains, branches, precision, and significant implicit choices. Measure semantic-reading accuracy rather than assuming a seal is understood. |
| S8 | What artifacts support maintenance? | Retain terms or certificates, source binding, dependencies, and useful method structure. Test upgrades and repair; replay levels are not interchangeable. |
| S9 | How should definitions be supported? | Package semantic definitions, characterizing theorems, presentations and observations as ordinary Lean assets. Pilot formal roots and EGF observations before a broad schema language. |
| S10 | What remains visible in a published document? | Every accepted assertion needs retained evidence. Default display must communicate semantic restrictions; proof details may be expandable. Reader studies remain necessary. |
| U1 | Is the first product a document view or a new input form? | Keep both as separable frontends to one backend. Compare the rendered-view and authoring effects rather than deciding from syntax samples. |
| U2 | What does an actual routine profile contain? | Name concrete solvers, simplification sets, budgets and versions. The first prototype must publish its profile and record failures, not just describe the abstraction. |
| U3 | What is the right unit for analytic plumbing? | Start with a composite local derivative theorem plus reusable lower-level lemmas. Let a tactic select and instantiate it; compare costs on the symbolic-index case. |

| ID | Restated question | Current answer and remaining test |
|---|---|---|
| U4 | Do theorem annotations survive refactoring? | Stable wrappers and semantic role IDs help, but mappings must be checked after signature changes. Include controlled rename and hypothesis edits in maintenance tests. |
| U5 | When can a change notice be dismissed? | After validating the resulting statement and affected policy or explanation, not merely because elaboration succeeds. Avoid asking about routine candidate formulas under fixed requests. |
| U6 | Is replay based on scripts, terms, or compiled files? | Retain a clear authority at a pin, normally terms or certificates, with scripts for editable reconstruction. Compiled caches do not alone demonstrate source migration. |
| U7 | What do provider outcomes and abstraction lists mean? | Distinguish unknown, qualified calculus outcomes, and original-target proof or refutation. An empty abstraction list is not an adequacy theorem. |
| U8 | How much value comes from structural adapters? | Use exact-target reconstruction as the acceptance boundary. Evaluate each adapter by coverage, cost, and its transport theorem; do not infer correctness from shape similarity. |
| U9 | What data should publication retain? | Retain the accepted evidence, original interpretation, dependencies, and replay policy. Alternative discovery traces are optional unless needed to support the published explanation. |
| U10 | Should definition support begin with linting, templates, or full syntax? | Begin with ordinary packages and a narrow template where it removes repeated obligations. Add syntax only after the equally equipped Lean comparison. |
| U11 | Who owns methods and resolves package policy conflicts? | Logical contracts belong to versioned packages; project policy chooses priorities and visibility. Conflicts require deterministic resolution and inspectable provenance. |
| U12 | When should language work stop at a library and linter? | Stop when the frontend adds no practical benefit after matching services and charging infrastructure cost, or when it harms semantic comprehension and maintenance. |

# C Validation, reproducibility, and limits

The source register records 70 tracked files in the nine input packages, with their SHA-256 hashes and Git blob identities at the imported commit. The input PDFs have respectively 37, 38, 37, 37, 41, 37, 36, 38, and 43 pages in the report order used throughout this document. Reading PDF metadata checks file identity and page counts; it does not establish that an input PDF was built from its paired TeX. The earlier two syntheses are registered separately as provenance.

The three reading memos contain detailed assessments and companion reproduction receipts. The synthesis's additional exact-arithmetic script checks six groups of mathematical examples: reversed telescoping bounds, a nonunit residual over $\mathbb{Z}/4\mathbb{Z}$, differentiation precision, the integer/rational ideal example, the five displayed formal-root coefficients, and the first four derivative polynomials. These are finite sanity checks supporting the review, not proofs of the generic theorems or an implemented language.

The isolated Lean experiment checks eight declarations concerning guarded real cancellation and the fixed-index binomial counterexample. It exited successfully under Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, using cached mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`. The reported axiom dependencies are `propext`, `Classical.choice`, and `Quot.sound`; no new axiom or native-computation axiom appears in those audits. The source, exact command, environment receipt, output, and reproduction instructions are retained under `evidence/lean`. This is focused kernel evidence for those statements, not formalization of Proposition 1, a CAS checker, Leant integration, or an aggregate repository build.

The new TeX and PDF are built together using three serial strict pdfLaTeX passes. The build script rejects unresolved references, missing glyphs, and overflowing boxes, and records source and PDF hashes. The final PDF is rendered for visual review, with document-level validation recorded in `validation.json`. The adjacent README explains reproduction. No source report or external repository is changed by this synthesis. The evidentiary boundary remains explicit: a reproducible report, nine reproduced companion programs, finite mathematical checks, and eight focused Lean declarations; no measured language productivity result or end-to-end language implementation is claimed.

## References


[1] *Accord*. Second-round proposal, imported 6 September 2026. Report PDF; `../ideas/accord/accord.tex`.

[2] *Cadence*. Second-round proposal. Report PDF; `../ideas/cadence/cadence.tex`.

[3] *Concord*. Second-round proposal. Report PDF; `../ideas/concord/concord.tex`.

[4] *Facet*. Second-round proposal. Report PDF; `../ideas/facet/facet.tex`.

[5] *Locus*. Second-round proposal. Report PDF; `../ideas/Locus_Proof_Language_Proposal/locus.tex`.

[6] *Meridian*. Second-round proposal. Report PDF; `../ideas/meridian/meridian.tex`.

[7] *Noema*. Second-round proposal. Report PDF; `../ideas/noema/noema.tex`.

[8] *Prism*. Second-round proposal. Report PDF; `../ideas/prism/prism.tex`.

[9] *Vantage*. Second-round proposal. Report PDF; `../ideas/vantage/vantage.tex`.

[10] Codex. *Mathematical Ideas, Checked Evidence: A Unified Evaluation of Nine Proof-Language Proposals*. First-round synthesis, 5 September 2026. Report PDF.

[11] Claude. First-round unified report, including the detailed Leant study and questions added before the second-round import. Report PDF.

[12] John Harrison and Laurent Théry. *A Skeptic's Approach to Combining HOL and Maple*. Journal of Automated Reasoning 21 (1998), 279–294. https://www.cl.cam.ac.uk/~jrh13/papers/cas.html.

[13] Lean Language Reference. *Validating Proofs*. Accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/ValidatingProofs/.